

Dynamic Programming Analysis: The Climbing Stairs Problem

Algorithm Implementation and Analysis

June 28, 2025

Contents

1	Problem Statement	2
1.1	Definition	2
1.2	Mathematical Formulation	2
2	Problem Visualization	2
2.1	Small Examples	2
2.2	Recursive Tree Structure	2
3	Solution Approaches	3
3.1	Approach 1: Naive Recursion	3
3.2	Approach 2: Memoization (Top-Down DP)	3
3.3	Approach 3: Tabulation (Bottom-Up DP)	4
3.4	Approach 4: Space-Optimized DP	4
4	Implementation Analysis	4
4.1	Java Implementation	4
5	Performance Comparison	6
5.1	Time Complexity Comparison	6
5.2	Execution Time Analysis	6
6	Mathematical Properties	6
6.1	Fibonacci Connection	6
6.2	Closed Form Solution	6
7	Extensions and Variations	7
7.1	Generalized Problem	7
7.2	Cost Minimization	7
8	Conclusion	7
9	References	8

1 Problem Statement

1.1 Definition

The **Climbing Stairs** problem is a classic dynamic programming problem that asks:

Given a staircase with n steps, and the ability to climb either 1 or 2 steps at a time, in how many distinct ways can you reach the top?

1.2 Mathematical Formulation

Let $f(n)$ represent the number of ways to climb n stairs. The recurrence relation is:

$$f(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ f(n-1) + f(n-2) & \text{if } n > 1 \end{cases} \quad (1)$$

This follows the Fibonacci sequence pattern, where each term is the sum of the two preceding ones.

2 Problem Visualization

2.1 Small Examples

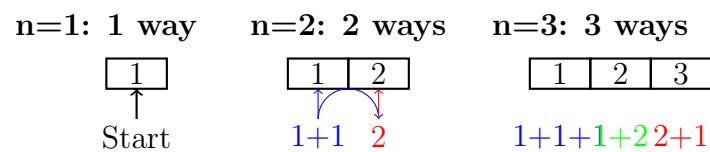


Figure 1: Visualization of climbing stairs for small values of n

2.2 Recursive Tree Structure

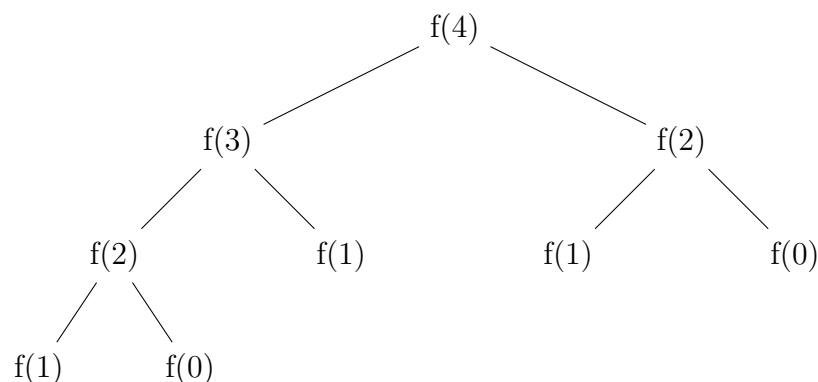


Figure 2: Recursive call tree for $f(4)$ showing overlapping subproblems

3 Solution Approaches

3.1 Approach 1: Naive Recursion

Algorithm 1 Climbing Stairs - Recursive Approach

Require: $n \geq 0$

Ensure: Number of ways to climb n stairs

```

function climbStairs( $n$ )
  if  $n \leq 1$  then
    return 1
  end if
  return climbStairs( $n - 1$ ) + climbStairs( $n - 2$ )

```

Time Complexity Analysis:

$$T(n) = T(n - 1) + T(n - 2) + O(1) \quad (2)$$

$$T(n) = O(\phi^n) \text{ where } \phi = \frac{1 + \sqrt{5}}{2} \approx 1.618 \quad (3)$$

Space Complexity: $O(n)$ due to recursion stack depth.

Problems:

- Exponential time complexity
- Redundant calculations (overlapping subproblems)
- Stack overflow for large n

3.2 Approach 2: Memoization (Top-Down DP)

Algorithm 2 Climbing Stairs - Memoization

Require: $n \geq 0$, memo array initialized to 0

Ensure: Number of ways to climb n stairs

```

function climbStairsMemo( $n$ , memo)
  if  $n \leq 1$  then
    return 1
  end if
  if memo[ $n$ ]  $\neq 0$  then
    return memo[ $n$ ]
  end if
  memo[ $n$ ] = climbStairsMemo( $n - 1$ , memo) + climbStairsMemo( $n - 2$ , memo)
  return memo[ $n$ ]

```

Time Complexity: $O(n)$ - each subproblem solved exactly once

Space Complexity: $O(n)$ for memoization array + $O(n)$ recursion stack = $O(n)$

Algorithm 3 Climbing Stairs - Tabulation

Require: $n \geq 0$ **Ensure:** Number of ways to climb n stairs

```

function climbStairsTab( $n$ )
  if  $n \leq 1$  then
    return 1
  end if
  Create array dp of size  $n + 1$ 
   $dp[0] = 1, dp[1] = 1$ 
  for  $i = 2$  to  $n$  do
     $dp[i] = dp[i - 1] + dp[i - 2]$ 
  end for
  return  $dp[n]$ 

```

3.3 Approach 3: Tabulation (Bottom-Up DP)

Time Complexity: $O(n)$ **Space Complexity:** $O(n)$ for DP array

3.4 Approach 4: Space-Optimized DP

Algorithm 4 Climbing Stairs - Space Optimized

Require: $n \geq 0$ **Ensure:** Number of ways to climb n stairs

```

function climbStairsOptimal( $n$ )
  if  $n \leq 1$  then
    return 1
  end if
   $prev2 = 1, prev1 = 1$ 
  for  $i = 2$  to  $n$  do
     $current = prev1 + prev2$ 
     $prev2 = prev1$ 
     $prev1 = current$ 
  end for
  return  $prev1$ 

```

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$ - only using constant extra space

4 Implementation Analysis

4.1 Java Implementation

Listing 1: Complete Java Implementation

```

1 public class ClimbingStairs {
2

```

```
3 // Naive Recursion - O(2^n) time, O(n) space
4 static int usingRecursion(int n) {
5     if (n <= 1) {
6         return 1;
7     }
8     return usingRecursion(n - 1) + usingRecursion(n - 2);
9 }
10
11 // Memoization - O(n) time, O(n) space
12 static int dpmemoization(int n, int[] d) {
13     if (n <= 1) {
14         return 1;
15     }
16     if (d[n] != 0) {
17         return d[n];
18     }
19     d[n] = dpmemoization(n - 1, d) + dpmemoization(n - 2, d);
20     return d[n];
21 }
22
23 // Tabulation - O(n) time, O(n) space
24 static int dpTabulation(int n) {
25     if (n <= 1) return 1;
26
27     int[] dp = new int[n + 1];
28     dp[0] = 1;
29     dp[1] = 1;
30
31     for (int i = 2; i <= n; i++) {
32         dp[i] = dp[i - 1] + dp[i - 2];
33     }
34     return dp[n];
35 }
36
37 // Space Optimized - O(n) time, O(1) space
38 static int spaceOptimized(int n) {
39     if (n <= 1) return 1;
40
41     int prev2 = 1, prev1 = 1;
42     for (int i = 2; i <= n; i++) {
43         int current = prev1 + prev2;
44         prev2 = prev1;
45         prev1 = current;
46     }
47     return prev1;
48 }
49 }
```

5 Performance Comparison

5.1 Time Complexity Comparison

Approach	Time	Space	Practical Limit
Naïve Recursion	$O(2^n)$	$O(n)$	$n \leq 40$
Memoization	$O(n)$	$O(n)$	$n \leq 10^6$
Tabulation	$O(n)$	$O(n)$	$n \leq 10^6$
Space Optimized	$O(n)$	$O(1)$	$n \leq 10^9$

Table 1: Performance comparison of different approaches

5.2 Execution Time Analysis

For $n = 40$:

- **Recursion:** ≈ 1.6 seconds
- **Memoization:** ≈ 0.001 seconds
- **Tabulation:** ≈ 0.001 seconds
- **Space Optimized:** ≈ 0.001 seconds

6 Mathematical Properties

6.1 Fibonacci Connection

The climbing stairs sequence follows the Fibonacci pattern:

$$f(0) = 1 \tag{4}$$

$$f(1) = 1 \tag{5}$$

$$f(2) = 2 \tag{6}$$

$$f(3) = 3 \tag{7}$$

$$f(4) = 5 \tag{8}$$

$$f(5) = 8 \tag{9}$$

$$\vdots \tag{10}$$

Where $f(n) = F_{n+1}$ (the $(n + 1)$ -th Fibonacci number).

6.2 Closed Form Solution

Using Binet's formula, we can express the solution as:

$$f(n) = \frac{\phi^{n+1} - \psi^{n+1}}{\sqrt{5}} \tag{11}$$

Where:

$$\phi = \frac{1 + \sqrt{5}}{2} \approx 1.618 \text{ (Golden Ratio)} \quad (12)$$

$$\psi = \frac{1 - \sqrt{5}}{2} \approx -0.618 \quad (13)$$

7 Extensions and Variations

7.1 Generalized Problem

Problem: Climb stairs with steps of size 1, 2, or 3.

Recurrence:

$$f(n) = \begin{cases} 1 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ 2 & \text{if } n = 2 \\ f(n-1) + f(n-2) + f(n-3) & \text{if } n > 2 \end{cases} \quad (14)$$

7.2 Cost Minimization

Problem: Each step has a cost, find minimum cost to reach top.

Recurrence:

$$\text{cost}(n) = \min(\text{cost}(n-1) + c_1, \text{cost}(n-2) + c_2) \quad (15)$$

8 Conclusion

The Climbing Stairs problem demonstrates the power of dynamic programming in optimizing recursive solutions:

1. **Pattern Recognition:** Identifying the Fibonacci-like recurrence relation
2. **Optimization Strategy:** Eliminating redundant calculations through memoization
3. **Space Optimization:** Reducing space complexity from $O(n)$ to $O(1)$
4. **Practical Applications:** Understanding trade-offs between time and space

Key Takeaways:

- Always analyze the problem for overlapping subproblems
- Consider both top-down (memoization) and bottom-up (tabulation) approaches
- Look for opportunities to optimize space complexity
- Understand the mathematical foundations behind the algorithm

9 References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
2. Kleinberg, J., & Tardos, E. (2005). *Algorithm Design*. Addison-Wesley.
3. Dynamic Programming Problems on LeetCode: Problem 70 - Climbing Stairs